

Reviewer Pack — Minimal Intersection Rank of Gröbner Bases for k-CLIQUE vs M...

Entry d4bf611f2558 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 07:45:58 UTC

Minimal Intersection Rank of Gröbner Bases for k-CLIQUE vs Monotone Circuit Depth

Entry ID: d4bf611f2558 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 07:45:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Gröbner Bases Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

{'line_1': 'The minimal intersection rank of the Gröbner basis for the k-CLIQUE problem is $\Theta(n^{\{2k/3\}})$.', 'line_2': 'For any monotone circuit computing k-CLIQUE, the depth of the circuit is at least $\Omega(n^k)$.', 'line_3': 'The ratio of these two complexities remains bounded away from zero.'}

Rationale (proposer's reasoning):

{'line_1': 'Gröbner bases are a fundamental tool in algebraic geometry and commutative algebra, known for their computational complexity.', 'line_2': 'k-CLIQUE is a well-known NP-complete problem that has been extensively studied for its circuit complexity.', 'line_3': 'This conjecture links the algebraic structure of Gröbner bases to the complexity of computing k-CLIQUE, potentially revealing new insights into both fields.'}

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d3cc4487babce0b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the ratio of the mean minimal intersection rank for k-CLIQUE to the mean monotone circuit depth exceeds an upper bound by at least 2 standard deviations with p-value < 0.05 , based on a minimum of 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Gröbner basis minimal intersection rank' AND 'monotone circuit depth'
- 'k-CLIQUE problem Gröbner basis' AND 'circuit complexity' - 'algebraic geometry Gröbner bases' AND 'complexity theory monotone circuits'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_cnf(n, k):
        if n < k or k <= 0:
            return []
        variables = list(range(1, n + 1))
        clauses = set()
        for _ in range(k):
            clause = random.sample(variables, k)
            clauses.add(tuple(sorted(clause)))
        return list(clauses)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        rank = 0
        for col in range(cols):
            pivot_row = -1
            for row in range(rank, rows):
                if matrix[row][col] != 0:
                    pivot_row = row
                    break
            if pivot_row == -1:
                continue
            matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]
            rank += 1
            for other_row in range(rank, rows):
                factor = -matrix[other_row][col] / matrix[pivot_row][col]
                for j in range(cols):
```

```

        if matrix[pivot_row][j]:
            matrix[other_row][j] += factor * matrix[pivot_row][j]
    return rank

def compute_minimal_intersection_rank(cnf):
    n = len(cnf)
    variables = set()
    for clause in cnf:
        variables.update(clause)
    m = len(variables)
    matrix = [[0] * (m + 1) for _ in range(m)]
    for i, var in enumerate(variables):
        for clause in cnf:
            if var in clause:
                matrix[i][clause.index(var)] = 1
    return gaussian_elimination(matrix)

def construct_monotone_circuit(cnf):
    n = len(cnf)
    variables = set()
    for clause in cnf:
        variables.update(clause)
    m = len(variables)
    circuit = []
    for var in variables:
        gate = random.choice(['AND', 'OR'])
        inputs = random.sample(list(variables - {var}), random.randint(1, m-2))
        circuit.append((gate, var, inputs))
    return circuit

def compute_circuit_size(circuit):
    size = 0
    for gate in circuit:
        if gate[0] == 'AND' or gate[0] == 'OR':
            size += len(gate[2])
        else:
            size += 1
    return size

n_values = [5, 10, 15, 20, 30, 40]
total_minimal_intersection_rank = 0
total_circuit_size = 0
instances_tested = 0

for n in n_values:
    for _ in range(5):
        k = random.randint(1, min(n-1, 5))
        cnf = generate_random_cnf(n, k)
        if not cnf:
            continue
        minimal_intersection_rank = compute_minimal_intersection_rank(cnf)
        circuit = construct_monotone_circuit(cnf)
        circuit_size = compute_circuit_size(circuit)
        total_minimal_intersection_rank += minimal_intersection_rank
        total_circuit_size += circuit_size
        instances_tested += 1

```

```

if instances_tested == 0:
    return {
        "metric_name": "minimal_intersection_rank_to_circuit_depth_ratio",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "no_valid_instances"
    }

mean_minimal_intersection_rank = total_minimal_intersection_rank / instances_tested
mean_circuit_size = total_circuit_size / instances_tested

if mean_minimal_intersection_rank == 0 or mean_circuit_size == 0:
    return {
        "metric_name": "minimal_intersection_rank_to_circuit_depth_ratio",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "zero_mean"
    }

ratio = mean_minimal_intersection_rank / mean_circuit_size
std_deviation = math.sqrt((sum((x - ratio) ** 2 for x in [mean_minimal_intersection_rank / mea
upper_bound = ratio + 2 * std_deviation

return {
    "metric_name": "minimal_intersection_rank_to_circuit_depth_ratio",
    "metric_value": ratio,
    "instances_tested": instances_tested,
    "conjecture_holds": ratio > upper_bound,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        results.append(trial_result)
        print(f"TRIAL: {trial_result}")

    total_minimal_intersection_rank = sum(r["metric_value"] for r in results if r["metric_value"]
    total_circuit_size = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
    instances_tested = sum(r["instances_tested"] for r in results)

    mean_minimal_intersection_rank = total_minimal_intersection_rank / instances_tested
    mean_circuit_size = total_circuit_size / instances_tested

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_minimal_intersection_rank} std={std_deviation} support
    elif sum(r["conjecture_holds"] for r in results) / len(results) >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_minimal_intersection_rank} std={std_deviation} support
    else:

```

```

first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
print(f"RESULT: FALSIFIED counterexample=\"minimal_intersection_rank_to_circuit_depth_ratio\")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39ee99a8.py", line 143, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39ee99a8.py", line 98, in run_trial
    circuit = construct_monotone_circuit(cnf)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39ee99a8.py", line 73, in construct_monotone_circuit
    inputs = random.sample(list(variables - {var}), random.randint(1, m-2))
                                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes without crashing and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14484
2	preregistration	ollama_remote	glm4:latest	0	0	11081
3	novelty	ollama_remote	glm4:latest	0	0	8534
4	novelty	ollama_remote	glm4:latest	0	0	9168
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16311
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11963

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10437
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16488
9	judge	ollama_remote	glm4:latest	0	0	11420

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109888 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/d4bf611f2558.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d4bf611f2558.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d4bf611f2558.tar.gz (if generated)